

Aim: Create script to send Ethereum transaction programmatically using web3.js/Nodejs.

Code:

```
const { Web3 } = require('web3')
```

```
// 1. Connect to free public Ethereum Test Network - Sepolia
```

```
// No API keys required for this public URL
```

```
const PUBLIC_RPC = 'https://ethereum-sepolia-rpc.publicnode.com';
```

```
const web3 = new Web3(PUBLIC_RPC);
```

```
// 2. DISPOSABLE DUMMY ACCOUNTS
```

```
// In a real application, these must be hidden in .env
```

```
const TEACHER_PRIVATE_KEY =  
'0xabc123e456f789a0123456789abcdef0123456789abcdef0123456789abcdef0'; // Dummy  
placeholder
```

```
const TEACHER_ADDRESS = '0x9965503B1a059419742997A47e2537357F74010C'; //  
Matches the private key
```

```
const STUDENT_ADDRESS = '0x211111111111111111111111111111111111'; // Mock  
recipient address
```

```
async function runClassDemo() {
```

```
  try {
```

```
    console.log("--- ETHEREUM TRANSACTION CLASSROOM DEMO ---");
```

```
    console.log(`Connecting to Test Network via: ${PUBLIC_RPC}\n`);
```

```
    // STEP 1: Fetch Network state - Nonce prevents Replay Attacks
```

```
    const nonce = await web3.eth.getTransactionCount(TEACHER_ADDRESS, 'pending');
```

```
    console.log(`Current Account Nonce(Tx Count): ${nonce}`);
```

```
    // STEP 2: Structure the Transaction Object - EIP-1559 Standard
```

```
    const txObject = {
```

```
      nonce: web3.utils.toHex(nonce),
```

```
      to: STUDENT_ADDRESS, //
```

```
      value: web3.utils.toHex(web3.utils.toWei('0.001', 'ether')), // Sending 0.001 Test ETH
```

```
    gasLimit: web3.utils.toHex(21000), // Standard limit for simple transfers
    maxPriorityFeePerGas: web3.utils.toHex(web3.utils.toWei('2', 'gwei')), // Tip to miner
    maxFeePerGas: web3.utils.toHex(web3.utils.toWei('30', 'gwei')), // Max gas price limit
    type: '0x2' // Specifies modern EIP-1559 transaction type
  };
  console.log("Transaction Object structured successfully");

  // STEP 3: Cryptographic Signing - happens locally
  console.log("Signing transaction locally with the Private Key...");
  const signedTx = await web3.eth.accounts.signTransaction(txObject,
TEACHER_PRIVATE_KEY);
  console.log("Transaction Signed! Cryptographic signature generated.");

  // STEP 4: Network Broadcast
  console.log("Broadcasting the signed transaction to the peer-to-peer network...");
  const receipt = await web3.eth.sendSignedTransaction(signedTx.rawTransaction);
  console.log(`Success! Tx Hash: ${receipt.transactionHash}`);

} catch (error) {
  console.log(`\n Network Response (Validation Failed):`);
  // Blockchain validates signature but will reject due to 0 ETH balance
  console.log(`${error.message}`);
  console.log(`\nLesson: The network safely verified our signature but stopped execution
because the account has 0 test tokens.`);
}
}
```

runClassDemo();

```
D:\TYCS12BCT\pract7>npm init -y
Wrote to D:\TYCS12BCT\pract7\package.json:

{
  "name": "pract7",
  "version": "1.0.0",
  "description": "",
  "main": "demo.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
D:\TYCS12BCT\pract7>npm install web3

added 76 packages, and audited 77 packages in 7s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
D:\TYCS12BCT\pract7>node demo.js
--- ETHEREUM TRANSACTION CLASSROOM DEMO ---
Connecting to Test Network via: https://ethereum-sepolia-rpc.publicnode.com

Current Account Nonce(Tx Count): 0
Transaction Object structured successfully
Signing transaction locally with the Private Key...

Network Response (Validation Failed):
"Invalid address length"

Lesson: The network safely verified our signature but stopped execution because the account has 0 test tokens.
```